

# MatMul or No MatMul in the Era of 1-bit LLMs

Jinendra Malekar, Mohammed E. Elbittity, Ramtin Zand

Computer Science and Engineering, University of South Carolina, Columbia, SC 29201  
jmalekar@email.sc.edu, elbittity@ieee.org, ramtin@cse.sc.edu

## Abstract

The advent of 1-bit large language models (LLMs) has attracted considerable attention and opened up new research opportunities. However, 1-bit LLMs only improve a fraction of models by applying extreme quantization to the projection layers while leaving attention heads unchanged. Therefore, to avoid fundamentally wrong choices of goals in future research, it is crucial to understand the actual improvements in computation and memory usage that 1-bit LLMs can deliver. In this work, we present an adaptation of Amdahl’s Law tailored for the 1-bit LLM context, which illustrates how partial improvements in 1-bit LLMs impact overall model performance. Through extensive experiments, we uncover key nuances across different model architectures and hardware configurations, offering a roadmap for future research in the era of 1-bit LLMs.

## Introduction

Generative Large language models (LLMs) such as GPT (Radford et al. 2019), OPT (Zhang et al. 2022) and LLaMA (Touvron et al. 2023) have attracted significant attention in recent years because of their impressive performance across various tasks, including but not limited to machine translation (Xu et al. 2024a), conversational chatbots (Sánchez Cuadrado et al. 2024), question answering (Tan et al. 2023), and even code generation (Ugare et al. 2024).

However, the remarkable performance of LLMs has come with increasing computational and energy costs. This necessitates expanding high-performance computing resources in data centers, potentially delaying green energy commitments and causing adverse environmental impacts (Stojkovic et al. 2024). Consequently, recent research has focused on optimizing the energy footprint of LLMs through various model optimization and compression techniques like pruning (Ma, Fang, and Wang 2023) and quantization (Xiao et al. 2023; Shao et al. 2023; Ma et al. 2024).

In addition to enhancing the energy efficiency of running LLMs in data centers, model optimization and compression can facilitate the deployment of LLMs on mobile and edge computing devices for real-time processing (Reidy et al. 2023). This advancement could benefit various emerging applications, including social robotics (Addlesee et al. 2024; Esteban-Lozano, Castro-González, and Martínez 2024), and augmented and virtual reality (Asadi et al. 2024; Morales

and Showalter-Bucher 2023).

Among LLM compression approaches, quantization has become a focal point, driven by works such as Smoothquant (Xiao et al. 2023), Omniquant (Shao et al. 2023), and more recently ShiftAddLLM (You et al. 2024) which emphasize post-training quantization (PTQ) as a cost-effective approach avoiding the processes of retraining and fine-tuning which can be particularly costly for LLMs.

A newly emerging approach to optimizing and compressing models through quantization-aware training (QAT) involves the extreme quantization of certain portions of LLMs, using binary  $\{-1, 1\}$  and ternary  $\{-1, 0, 1\}$  weights (Wang et al. 2023; Ma et al. 2024). This development has initiated the era of 1-bit LLMs. Besides the memory utilization advantages, extreme quantization transforms the costly matrix multiplication (MatMul) operations into more efficient addition and subtraction operations, and thus leading to MatMul-free operations. However, it is important to note that not all MatMul operations can undergo extreme quantization due to the resulting drop in accuracy. Specifically, MatMul operations in the attention heads still require higher precisions, such as 16-bit floating point (FP16) or 8-bit integer (INT8). A follow-up work (Zhu et al. 2024a) has built upon BitNet (Wang et al. 2023), aiming to eliminate MatMul operations in attention heads by using Hadamard products.

The advent of 1-bit LLMs has paved the way for various new research directions. However, since they currently address only a fraction of the model, a crucial question arises:

*What effects do partial enhancements in 1-bit LLMs have on the overall performance of the model?*

Answering this question is vital for guiding future research effectively and avoiding misguided or ineffective goals. For example, if the MatMul operations that are replaced with MatMul-free operations in current 1-bit LLMs account for the majority of the model’s computation and memory usage, then focusing on optimizing the relatively minor MatMul operations in attention heads may be less impactful. In this case, prioritizing custom hardware development to fully leverage extreme quantization would be more sensible. Conversely, if the conversion of the fraction of MatMul operations to MatMul-free operations in current 1-bit LLMs does not significantly affect overall computation and memory usage, it would be more prudent to fo-

cus on optimizing the MatMul operations in the attention heads—currently not optimized in 1-bit LLMs—rather than investing in hardware development for current 1-bit LLMs.

In this study, we address the highlighted research question through extensive experiments and analyses on various LLMs. We explore various model hyperparameters across two types of hardware designed for edge and cloud environments. In addition, we propose an adaptation of Amdahl’s Law for LLMs to identify how partial improvements in LLM can translate into overall enhancements for the entire model. Our findings reveal important nuances that can guide future research in the 1-bit LLMs era.

## Related Work

### Transformer Quantization

Transformer quantization can be categorized into weight-only and weight-and-activation quantization. Weight-only quantization reduces memory requirements, while weight-and-activation quantization also enhances computational efficiency. SmoothQuant (Xiao et al. 2023) supports both activation and weight quantization, based on the principle that the difficulty of activation quantization can be mathematically transformed. This method demonstrates a  $1.5\times$  speedup and  $2\times$  memory reduction for LLMs with negligible loss, supporting W8A8 (8-bit weight, 8-bit activation) quantization. Omniquant (Shao et al. 2023) is another method supporting a broader spectrum of weight-activation quantization configurations, including W4A4, W4A16, W3A16, and W2A16. This is achieved through learnable weight clipping, which optimizes the clipping threshold, and learnable equivalent transformation, which mitigates activation outliers by shifting them to weights. LLM.int8() (Dettmers et al. 2022) loads the model in 8-bit format, with 99.9% of operations performed in 8-bit, except for emergent outliers. It uses vector-wise quantization with different normalization constants to preserve model performance. GPTQ (Frantar et al. 2023) focuses solely on weight quantization, achieving 3-bit and 4-bit quantization with speedups of  $3.24\times$  and  $4.53\times$  on A6000 and A100 GPUs. QuIP# (Tseng et al. 2024) is a weight-only quantization method that achieves state-of-the-art performance in sub-4-bit quantization. It employs a randomized Hadamard transform combined with vector quantization and then fine-tunes the model to enhance its fidelity. All the methods discussed above are related to PTQ.

### Era of 1-bit LLMs

Before the LLM era, the concept of binary weight and activation quantization was explored in works such as Binarized Neural Networks (BNN) (Hubara et al. 2016) and Ternary Neural Networks (TNN) (Alemdar et al. 2017). While these studies did not focus on generative language tasks, they achieved significant performance improvements, with BNNs demonstrating up to  $7\times$  performance gains and TNNs showing up to  $3.1\times$  energy efficiency improvements. These findings underscore the remarkable ability of neural networks to operate effectively with just 1-bit precision.

Recently, following the rise of LLMs and advancements in model performance, the BitNet (Wang et al. 2023) introduced a 1-bit transformer quantization technique for LLMs. This method replaces the conventional ‘nn.Linear’ layer in PyTorch with a new BitLiner layer, where weights are restricted to either 1 or -1, and activations are represented in 8-bit precision. Despite this quantization, other components like self-attention remain in 8-bit format. BitNet’s design suggests that it can scale to even larger transformer models, following scaling laws similar to those used for full-precision transformers. A variant of BitNet, known as BitNet 1.58 (Ma et al. 2024), employs ternary weights (-1, 0, 1) and achieves perplexity and end-task performance comparable to full-precision transformers (FP16 or BF16). In terms of the computation, 1-bit LLMs transform MatMul operations into addition operations, due to the 1-bit nature of the weights, except for layers like attention, which need to remain in high precision to maintain performance. Additionally, 1-bit LLMs address the challenge of transferring model parameters from DRAM to on-chip accelerators such as SRAM, prompting the development of architectures optimized for the efficient operation of 1-bit LLMs.

## Approach

### Demystifying the Underlying Operations of LLMs

The overall architecture of the generative LLMs is shown in Figure 1, which includes  $N$  decoder blocks, each consisting of self-attention and feedforward layers followed by add and normalization operations (Vaswani et al. 2017). The core of the LLM is the self-attention mechanism with multiple heads ( $h$ ). For an attention head, the computation begins with three linear projections of the token vector to form the Key ( $K$ ), Query ( $Q$ ), and Value ( $V$ ) vector sequences:

$$Q = W_Q * I, \quad K = W_K * I, \quad V = W_V * I \quad (1)$$

where  $I$  is the input token vector and  $W_Q$ ,  $W_K$ , and  $W_V$  are trainable weight matrices with  $d \times d$  dimensions where  $d$  is the embedding dimension. The operator  $*$  denotes the MatMul operation.

The generated  $K$ ,  $Q$ , and  $V$  vectors are then divided into  $h$  vectors with reduced dimensionality of  $d/h$  where  $h$  is the number of attention heads. At this stage, the value and key vectors are concatenated with the previous  $l - 1$  value and key tokens that are cached from previous token generation iterations to form a  $d/h \times l$  matrix in each head, where  $l$  is the sequence length. Next, the attention scores are computed using the scaled dot-product of the queries and keys multiplied with the generated value matrix (Vaswani et al. 2017). This step includes two MatMul ( $*$ ) operations:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q * K^T}{\sqrt{d}}\right) * V \quad (2)$$

Subsequently, the output of the attention heads are concatenated and linearly transformed as follows:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) * W_X \quad (3)$$

where  $\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$ , and  $W_X$  is a  $d \times d$  matrix with trainable elements.

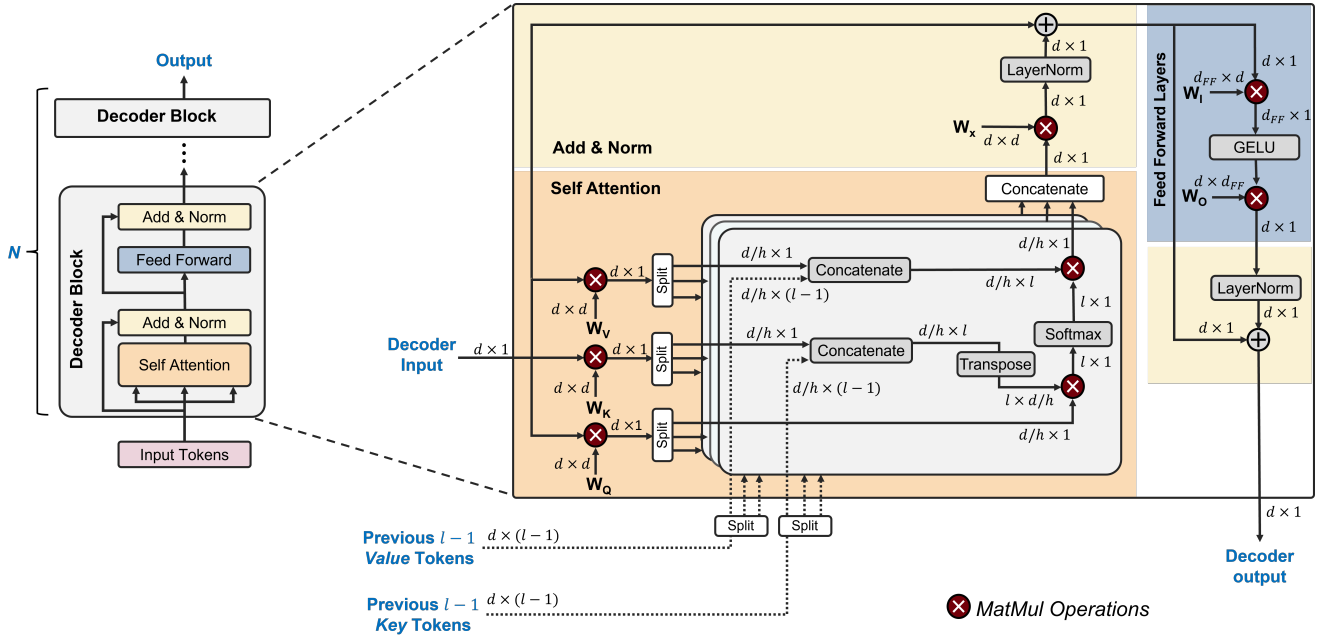


Figure 1: The typical architecture of decoder-only LLM models and its underlying operations. The tokenization and embedding layers are not shown in the figure.

| Block                 | Description     | Dimension                               |
|-----------------------|-----------------|-----------------------------------------|
| Attention Projections | $W_Q$           | $(d \times d) * (d \times 1)$           |
|                       | $W_K$           |                                         |
|                       | $W_V$           |                                         |
|                       | $W_X$           |                                         |
| Attention Head        | $Q * K$         | $(l \times d/h) * (d/h \times 1)$       |
|                       | $V * Score$     | $(d/h \times l) * (l \times 1)$         |
| FFN                   | Intermediate FF | $(d_{FF} \times d) * (d \times 1)$      |
|                       | Output FF       | $(d \times d_{FF}) * (d_{FF} \times 1)$ |

Table 1: MatMul Operations and their dimensions in decoder-only LLMs. The parameters  $d$ ,  $h$ ,  $l$ , and  $d_{FF}$  denote embedding dimension, number of attention heads, sequence length, and the size of the feed-forward layers, respectively.

Finally, the attention output is followed by a feed-forward network (FFN) that involves two linear and one nonlinear transformations. As shown in Figure 1, Gaussian error linear unit (GELU) (Hendrycks and Gimpel 2016) activation function is frequently utilized in LLMs to introduce nonlinearity into FFN. Therefore, the FFN computation can be described as follows, which involves two more MatMul operations:

$$FFN(x) = GELU(x * W_I + b_I) * W_O + b_O \quad (4)$$

where  $W_I$  and  $W_O$  are trainable matrices with  $d_{FF} \times d$  and  $d \times d_{FF}$  dimensions, respectively, where  $d_{FF}$  is the feed-forward layer size. Also, the  $x$  is MultiHead( $Q, K, V$ ). Both the self-attention and FFN layers are followed by layer normalization which can also introduce some nonlinearity in the process of division by the standard deviation.

In summary, the computation of decoder blocks in LLMs involves a combination of nonlinear operations (Layer-

Norm, softmax, GELU), and linear MatMul operations. In particular, there are a total of  $2h + 6$  MatMul operations in a decoder block including six weight-to-activation MatMuls ( $W_Q$ ,  $W_K$ ,  $W_V$ ,  $W_X$ ,  $W_I$ , and  $W_O$  projections) plus  $2h$  activation-to-activation MatMuls to compute  $Attention(Q, K, V)$  in each head ( $h$ ). In decoder-only LLMs, MatMuls are matrix-vector multiplications since the inference is performed iteratively, processing one input token per iteration and the keys and values from previous iterations are cached as shown in Figure 1. Table 1 lists the dimensions of each of these MatMul operations.

Previous works (Kim et al. 2023) have shown that designing dedicated hardware to compute nonlinear operations in LLMs can make their computation overhead negligible compared to MatMul operations. Consequently, optimizing MatMul operations can lead to a significant speedup in LLM computation. The 1-bit LLMs (Wang et al. 2023; Ma et al. 2024; Xu et al. 2024b) as a recent approach that has attracted considerable attention involves extreme quantization of MatMuls in LLMs, except for the attention heads, which require higher precision to maintain accuracy. Quantizing all the weight matrices ( $W_Q$ ,  $W_K$ ,  $W_V$ ,  $W_X$ ,  $W_I$ , and  $W_O$ ) in the LLMs to include only binary  $\{-1, 1\}$  or ternary  $\{-1, 0, 1\}$  elements transforms the weight-to-activation MatMul operations to simple addition and subtraction operations, dividing the LLM models into two portions one with MatMul and one without MatMul (or MatMul-free), as shown in Figure 2.

Due to variations in the model hyperparameters ( $d$ ,  $l$ ,  $h$ , and  $d_{FF}$ ) across different types of LLMs, the proportion of the linear projections (weight-to-activation MatMuls) and attention head (activation-to-activation MatMuls) computation relative to the entire model can vary significantly. Therefore, performance analysis with layer-wise granularity, tar-

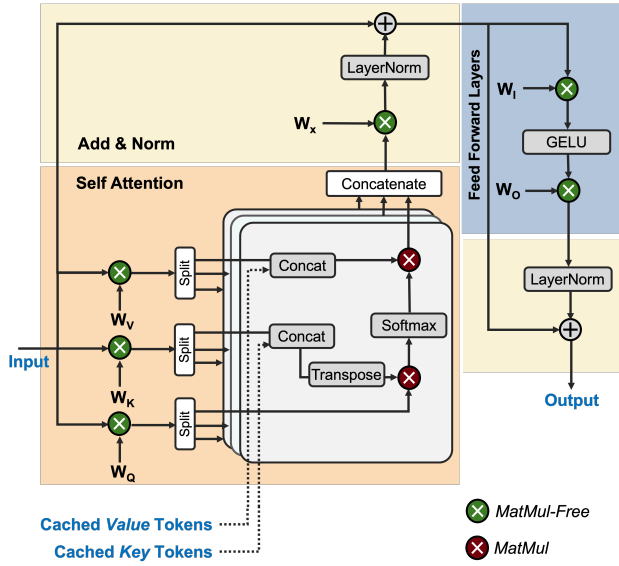


Figure 2: The 1-bit LLMs divide the model into two portions: attention heads with MatMul operations (shown in red) and MatMul-free projection layers (shown in green).

geted in this work, can illuminate the effectiveness of the 1-bit LLMs and help determine future research directions.

### Design of LLM-Specific Hardware

For the performance analysis, we leverage tensor processing unit (TPU) architectures which are specifically designed to accelerate the MatMul operations dominant in machine learning workloads. TPUs maximize data reuse while minimizing data transfer by utilizing systolic arrays at their core (Jouppi 2017; Jouppi et al. 2017).

A systolic array typically consists of two-dimensional arrays of processing elements (PEs). Each PE performs a multiply-and-accumulate (MAC) operation, multiplying weights and inputs using a multiplier circuit and adding the result to previously computed partial sums using an accumulator circuit. The MAC result is either retained within the same PE or broadcast to other PEs for further computations, depending on the dataflow architecture. This MAC operation is executed in every PE of the systolic array, enabling efficient MatMul operations by maximizing data reuse and minimizing additional data transfer overhead.

The dataflow in a systolic array is a mapping scheme determined by the microarchitecture of the PEs, which dictates how input data is fed into the array and how partial results and outputs are generated and stored. Rather than repeatedly loading and storing data to and from memory, each PE typically follows one of these dataflow architectures: (1) *Input Stationary (IS)*: The inputs (or activations) remain fixed in the PEs while the weights are sequentially fed into the PEs; (2) *Output Stationary (OS)*: The outputs are attached with the MAC units, while the inputs and weights circulate among the PEs. Inputs and weights are loaded, multiplied, and the results are accumulated with partial sums held in the PE. (3) *Weight Stationary (WS)*: Each weight is preloaded

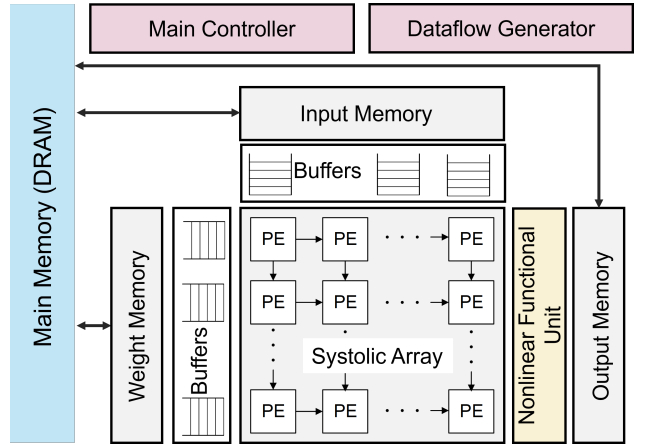


Figure 3: The overall architecture of the TPU that is designed for accelerating LLMs, featuring dedicated hardware to support nonlinear operations.

into a register within each PE. During each cycle, inputs are multiplied by the weights and broadcast across the PEs.

Figure 3 shows the architecture of TPU, consisting of weight, input, and output memories, and a systolic array of size  $S = N \times N$  PEs surrounded by the first-in-first-out (FIFO) buffers. Additionally, our TPU includes a *Nonlinear Functional Unit*, featuring custom hardware to support nonlinear operations in the LLMs. The *Dataflow Generator* block generates the memory read/write addresses to store or retrieve the inputs, weights, and outputs according to the selected dataflow. The *Main Controller* manages the data transfer between memories, FIFOs, and the systolic array.

As previously discussed, the MatMul operations in generative LLMs involve matrix-vector multiplications. Consequently, the sizes of the input and output vectors are always smaller than those of the weight matrices. For activation-to-activation MatMuls in the attention head, where there are no weight values (See Figure 1), we store the concatenated *Value* and *Key* matrices (with  $d/h \times l$  and  $l \times d/h$  dimensions, respectively) in the weights memory, while the *Query* and *attention score* vectors are stored in the input memory. Based on the pattern in the size of the input, weight, and output tensors in matrix-vector multiplications involved in LLMs (mentioned in Table 1), a TPU design with larger weight memory compared to input and output memories would be more efficient, as it reduces the need for costly accesses to the main DRAM memory to load the weights.

For the dataflow architecture, we conducted comprehensive experiments utilizing IS, OS, and WS dataflows. Based on the results obtained (refer to Appendix A), the OS dataflow architecture demonstrated the best performance. The OS dataflow is particularly advantageous for accumulating results since partial sums remain stationary and do not need to be moved frequently. Additionally, once weight and input values are fetched from their respective memories, they are reused by passing from one PE to another, leveraging the spatial dataflow capabilities of TPUs.



## Amdahl’s Law of LLMs

Since current 1-bit LLMs only improve a part of the model (the projections without enhancing the attention heads), we need a mechanism to determine how these partial improvements translate into overall enhancements for the entire model. This is crucial for addressing the main research question of the paper.

The Amdahl’s Law provides a framework for such scenarios. Amdahl’s Law is a formula used to find the maximum improvement in a system when only part of it is improved. It can be expressed as:

$$S_{total} = \frac{1}{1 - F + \frac{F}{S_{partial}}} \quad (5)$$

where  $F$  is the fraction of the system that is improved,  $S_{partial}$  is the factor by which the part  $F$  is improved, and  $S_{total}$  is the overall improvement of the entire system.

In the context of 1-bit LLMs, we define  $F$  as the fraction of the MatMul operations that can be replaced with MatMul-free operations by using extreme quantization, relative to all MatMul operations in the model. Here, our focus is on quantifying the value of  $F$  across various LLMs and hardware configurations. Enhancing  $S_{partial}$  is closely tied to the design of custom hardware accelerators for binary and ternary operations, which is beyond the scope of this paper.

## Experiments

### Simulation Setup

For our experiments, we study 13 different LLMs including GPT, OPT, and LLaMA models. Table 2 lists all models and their corresponding hyperparameters. To save space in the main body of the paper, we only provide the results for the seven OPT models, as they represent a diverse range of model hyperparameter combinations. The results for GPT and LLaMA models are provided in the Appendix B.

For the hardware, we designed two TPUs tailored for different applications: cloud and edge processing. The cloud TPU features a  $256 \times 256$  systolic array with 16MB of SRAM, while the edge TPU has a  $32 \times 32$  systolic array with 8MB of SRAM. Both systolic arrays employ an OS dataflow. Also, in both designs, 2MB of memory is allocated for internal use, including storing control and configuration data, tracking computation states, managing data flow, and ensuring seamless data movement. All memories use double buffering to mask the latency associated with SRAM access. Table 3 provides the memory distribution of both edge and cloud TPU designs. We utilize the cycle-accurate SCALE-Sim framework (Samajdar et al. 2018, 2020) to measure compute cycles and memory accesses in various LLMs.

Our experiments investigate the distribution of computation and memory utilization between the attention heads, which require MatMul operations, and the projections, which can be binarized or ternarized and consequently do not involve MatMuls. In the results presented, the terms “MatMul” and “MatMul-Free” refer to these respective parts of the LLM computation.

| Models | Param | Hyperparameters |     |          |          |
|--------|-------|-----------------|-----|----------|----------|
|        |       | $d$             | $h$ | $d_{FF}$ | $l$      |
| GPT    | 125M  | 768             | 12  | 768      | 128-4096 |
|        | 355M  | 1024            | 16  | 1024     | 128-4096 |
|        | 774M  | 1280            | 20  | 1280     | 128-4096 |
|        | 1.5B  | 1600            | 25  | 1600     | 128-4096 |
| OPT    | 350M  | 1024            | 16  | 4096     | 128-4096 |
|        | 1.3B  | 2048            | 32  | 8192     | 128-4096 |
|        | 2.7B  | 2560            | 32  | 10240    | 128-4096 |
|        | 6.7B  | 4096            | 32  | 16384    | 128-4096 |
|        | 13B   | 5120            | 40  | 20480    | 128-4096 |
|        | 30B   | 7168            | 56  | 28672    | 128-4096 |
|        | 66B   | 9216            | 76  | 36864    | 128-4096 |
| LLaMA  | 7B    | 4096            | 32  | 11008    | 128-4096 |
|        | 13B   | 5120            | 40  | 13824    | 128-4096 |

Table 2: Model Hyper-parameters used for simulations.

| Design | Systolic Array Size | Memory Capacity |        |        |
|--------|---------------------|-----------------|--------|--------|
|        |                     | Input           | Output | Weight |
| Cloud  | $256 \times 256$    | 4MB             | 4MB    | 8MB    |
| Edge   | $32 \times 32$      | 2MB             | 2MB    | 4MB    |

Table 3: The specification of the edge and cloud TPUs.

### Performance Analysis on Cloud Setup

In the first set of experiments, we examine various language models deployed on the cloud TPU setup to determine the fraction of the models that can become MatMul-Free in 1-bit LLMs, i.e., projection layers, through extreme quantization. To achieve this, we compare seven OPT models of various sizes (ranging from 350M to 66B parameters) with different sequence lengths (ranging from 128 to 4096). We vary the sequence length ( $l$ ) because it only affects the computation in the attention heads and determines the size of the remaining MatMul operations in 1-bit LLMs (refer to Table 1).

Figure 9 exhibits the fraction of MatMul-Free operations in the OPT models deployed on the cloud setup, measured in terms of compute cycles and memory accesses. The fraction of MatMul-Free operations varies from roughly **23%** to **98%** for compute cycles and from **59%** to **99.8%** for memory access across different configurations. **In general, MatMul-Free operations increase with model size and decrease with sequence length.**

**Compute Cycle Analysis.** Typically, the context length of the OPT models is equal to 2048. The MatMul-Free compute cycles for these OPT models can be observed in the 2048 row of Figure 9 (a). For smaller language models like OPT 350M, approximately 63% of the computation occurs in the attention heads, involving MatMul operations, while only 37.1% of the computation benefits from MatMul-Free operations provided by 1-bit LLMs. Moreover, the OPT 1.3B model with a sequence length of 2048 is particularly noteworthy, as half of its computation can be MatMul-Free, while the other half involves MatMul operations.

Models like LLaMA 7B and 13B generally use larger se-

|                 |      |              |              |              |              |             |             |             |
|-----------------|------|--------------|--------------|--------------|--------------|-------------|-------------|-------------|
| Sequence Length | 128  | 0.777        | 0.858        | 0.900        | 0.955        | 0.963       | 0.973       | 0.978       |
|                 | 256  | 0.759        | 0.845        | 0.891        | 0.950        | 0.959       | 0.970       | 0.976       |
|                 | 512  | 0.661        | 0.771        | 0.834        | 0.921        | 0.935       | 0.952       | 0.962       |
|                 | 1024 | 0.524        | 0.656        | 0.739        | 0.868        | 0.890       | 0.917       | 0.934       |
|                 | 2048 | 0.371        | 0.505        | 0.603        | 0.778        | 0.812       | 0.855       | 0.883       |
|                 | 4096 | 0.234        | 0.346        | 0.440        | 0.645        | 0.690       | 0.754       | 0.795       |
|                 |      | 1024<br>350M | 2048<br>1.3B | 2560<br>2.7B | 4096<br>6.7B | 5120<br>13B | 7168<br>30B | 9216<br>66B |

(a) Compute Cycles

|                 |      |              |              |              |              |             |             |             |
|-----------------|------|--------------|--------------|--------------|--------------|-------------|-------------|-------------|
| Sequence Length | 128  | 0.978        | 0.989        | 0.991        | 0.995        | 0.996       | 0.997       | 0.998       |
|                 | 256  | 0.958        | 0.979        | 0.983        | 0.990        | 0.992       | 0.994       | 0.995       |
|                 | 512  | 0.921        | 0.959        | 0.967        | 0.979        | 0.983       | 0.988       | 0.991       |
|                 | 1024 | 0.853        | 0.921        | 0.936        | 0.959        | 0.967       | 0.976       | 0.982       |
|                 | 2048 | 0.745        | 0.854        | 0.880        | 0.922        | 0.937       | 0.954       | 0.964       |
|                 | 4096 | 0.593        | 0.745        | 0.786        | 0.856        | 0.881       | 0.912       | 0.930       |
|                 |      | 1024<br>350M | 2048<br>1.3B | 2560<br>2.7B | 4096<br>6.7B | 5120<br>13B | 7168<br>30B | 9216<br>66B |

(b) Memory Access

Figure 4: Fraction of Matmul-free operations in the OPT models deployed on the cloud setup.

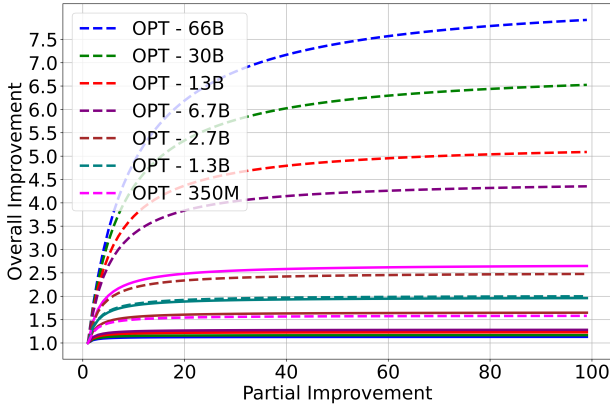


Figure 5: Amdahl’s Law of LLMs for cloud deployment scenario. The dashed lines and solid lines show the effect of partial improvement in projection layers and partial improvement in attention layers, respectively.

sequence lengths of 4096. In terms of model size, they are comparable to OPT 6.7B and 13B models. As illustrated in the 4096 row of Figure 9 (a), MatMul-Free operations account for 64.5% and 69% of the computation for OPT 6.7B and OPT 13B models, respectively. A detailed analysis of the LLaMA models can be found in the Appendix B.

**Memory Access Analysis.** 1-bit LLMs, such as BitNet (Wang et al. 2023) and BitNet 1.58 (Ma et al. 2024), can achieve up to  $16\times$  and  $8\times$  reductions in weight memory utilization compared to FP16 LLMs because they use just 1 bit and 2 bits to represent binary and ternary weights, respectively. Besides memory capacity, another crucial factor affecting LLM throughput is the number of memory accesses required for inference. Therefore, we also analyzed the memory access of the MatMul-Free and MatMul components of the 1-bit LLM architecture. Figure 9 (b) shows the ratio of memory reads and writes associated with the projection layers (MatMul-Free parts) to those of the entire model.

The results reveal a trend similar to the compute cycle analysis. However, unlike compute cycles, for all cases, the majority of memory accesses are associated with the projection layers. For example, for OPT models with a sequence length of 2048, the fraction of memory access for the OPT 350M is **74%**, increasing to **96%** for the OPT 66B. The memory access analysis suggests opportunities for research into hybrid memory hierarchy designs, where the components of the model dominating memory accesses can be off-loaded to faster memory technologies. For example, in OPT 6.7B with a sequence length of 4096, moving the projection layers’ weight and activation data to faster memory can speed up more than 85% of all memory accesses.

**Amdahl’s Law of LLMs.** Here, we leverage Amdahl’s Law of LLMs proposed in Equation (5) to show how partial improvements in the LLMs can enhance overall performance. In particular, we vary  $S_{partial}$  from 1 to 100, and using the fractions ( $F$ ) shown in Figure 9, calculate the overall improvement of model ( $S_{total}$ ). Figure 5 demonstrates an Amdahl’s Law analysis when improvements are applied to either attention layers (MatMul parts) or the projections layers (MatMul-Free parts) of LLMs. The dashed lines show the effects of improving the projection layers, while the solid lines represent the impact of improvements to the attention layers. This analysis is based on OPT models with a typical sequence length of 2048. For Amdahl’s Law analyses with other sequence lengths, please refer to the Appendix C.

The Amdahl’s Law analysis of LLMs deployed on the cloud setup reveals three key takeaways:

1. For smaller language models like OPT 350M, improving the attention heads has a greater impact than enhancing the projection layers. Therefore, using 1-bit LLM paradigms to improve these models may result in limited overall performance gains.
2. Medium-sized LLMs, such as OPT 1.3B and 2.7B, can benefit from combining 1-bit LLM improvements with enhancements to the attention heads, such as replacing MatMul operations with Hadamard products and additive operators as proposed in (Zhu et al. 2024b).

|                 |      |                     |              |              |              |             |             |             |
|-----------------|------|---------------------|--------------|--------------|--------------|-------------|-------------|-------------|
| Sequence Length | 128  | 0.967               | 0.983        | 0.986        | 0.992        | 0.994       | 0.996       | 0.997       |
|                 | 256  | 0.940               | 0.968        | 0.974        | 0.986        | 0.989       | 0.992       | 0.994       |
|                 | 512  | 0.890               | 0.941        | 0.951        | 0.974        | 0.979       | 0.985       | 0.988       |
|                 | 1024 | 0.805               | 0.890        | 0.909        | 0.950        | 0.960       | 0.971       | 0.977       |
|                 | 2048 | 0.677               | 0.804        | 0.835        | 0.906        | 0.923       | 0.944       | 0.956       |
|                 | 4096 | 0.512               | 0.673        | 0.718        | 0.829        | 0.858       | 0.894       | 0.916       |
|                 |      | 1024<br>350M        | 2048<br>1.3B | 2560<br>2.7B | 4096<br>6.7B | 5120<br>13B | 7168<br>30B | 9216<br>66B |
|                 |      | Embedding Dimension |              |              |              |             |             |             |

(a) Compute Cycles

|                 |      |                     |              |              |              |             |             |             |
|-----------------|------|---------------------|--------------|--------------|--------------|-------------|-------------|-------------|
| Sequence Length | 128  | 0.979               | 0.989        | 0.992        | 0.995        | 0.996       | 0.997       | 0.998       |
|                 | 256  | 0.959               | 0.979        | 0.983        | 0.990        | 0.992       | 0.994       | 0.995       |
|                 | 512  | 0.921               | 0.959        | 0.967        | 0.979        | 0.983       | 0.988       | 0.991       |
|                 | 1024 | 0.855               | 0.921        | 0.936        | 0.960        | 0.967       | 0.977       | 0.982       |
|                 | 2048 | 0.746               | 0.854        | 0.880        | 0.922        | 0.937       | 0.954       | 0.964       |
|                 | 4096 | 0.595               | 0.746        | 0.786        | 0.856        | 0.881       | 0.912       | 0.930       |
|                 |      | 1024<br>350M        | 2048<br>1.3B | 2560<br>2.7B | 4096<br>6.7B | 5120<br>13B | 7168<br>30B | 9216<br>66B |
|                 |      | Embedding Dimension |              |              |              |             |             |             |

(b) Memory Access

Figure 6: Fraction of Matmul-free operations in the OPT models deployed on the edge setup.

- For larger models like OPT 6.7B and above, improvements to the attention heads have a minimal effect on overall performance. In these cases, employing 1-bit LLM methods alone can lead to significant gains in performance and throughput.

### Performance Analysis on Edge Setup

Figure 10 presents the compute and memory analysis for the edge setup. In all cases, most of the computations occur in the MatMul-Free portion. The only instance where the computation is relatively evenly distributed between the MatMul and MatMul-Free parts is for OPT 350M with a 4096 sequence length. A similar pattern is observed in memory accesses. This happens because, in the edge setup, the smaller  $32 \times 32$  systolic array is less efficient at handling the larger matrices typically found in projection layers. Meanwhile, the MatMul operations in the attention heads are smaller due to the splitting of large matrices among multiple heads (refer to Figure 1), making these operations more manageable even with smaller systolic arrays. Consequently, this increases the ratio of computation in the projection layers compared to the attention heads.

**Amdahl’s Law of LLMs.** Figure 7 illustrates the Amdahl’s Law analysis for the OPT models with 2048 sequence length deployed on the edge TPU setup. The results indicate that, unlike in the cloud setup, across all cases, enhancing the projection layers leads to significantly greater overall improvements in the entire model. Conversely, improving the attention layers yields only marginal gains. This suggests that 1-bit LLM approaches targeting optimization of projection layers are significantly more efficient in the edge setup. Therefore, a promising research direction would be to focus on developing efficient custom hardware for implementing extremely quantized projection layers, rather than concentrating on algorithmic and hardware innovations to enhance computation in the attention heads.

### Conclusion

The proposal of 1-bit LLMs has opened several research avenues, including the development of custom hardware for 1-

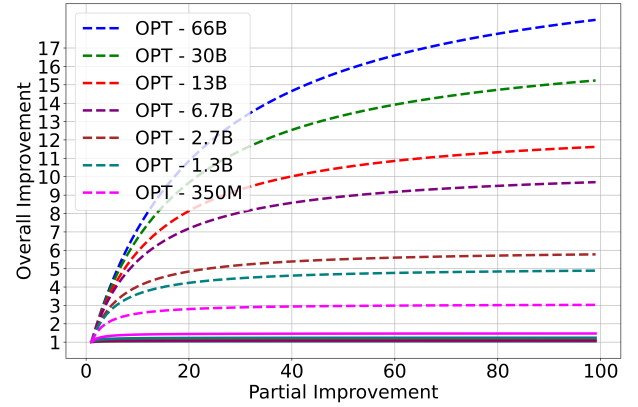


Figure 7: Amdahl’s Law of LLMs for edge deployment scenario. The dashed lines and solid lines show the effect of partial improvement in projection layers and partial improvement in attention layers, respectively.

bit LLMs as well as algorithmic innovations to enhance aspects of LLM computation that 1-bit LLMs cannot address. Here, we aimed to provide a roadmap to avoid fundamentally incorrect or inefficient research goals in this field. We introduced the concept of Amdahl’s Law of LLMs, which helps determine how partial improvements from 1-bit LLMs translate into overall model enhancements. We conducted extensive evaluations across different LLMs with various hyperparameters to identify relevant patterns. The results reveal a significant dependency of 1-bit LLM efficacy on model sizes, hyperparameters, and hardware configurations. Key findings include: (i) 1-bit LLM paradigms have limited impact on smaller language models, particularly when the context length is large, (ii) for medium-sized LLMs, 1-bit LLM methods show benefits, but further algorithmic innovations are needed to enhance parts that 1-bit LLM approaches cannot improve, and (iii) for large-scale LLMs, extreme quantization from 1-bit LLMs alone can improve the majority of computations, in some cases by more than 99%.

## Acknowledgments

This work is supported in part by the National Science Foundation (NSF) under grant numbers 2340249 and 2409697.

## References

- Addlesee, A.; Cherakara, N.; Nelson, N.; Hernández García, D.; Gunson, N.; Sieińska, W.; Romeo, M.; Dondrup, C.; and Lemon, O. 2024. A multi-party conversational social robot using LLMS. In *Companion of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, 1273–1275.
- Alemdar, H.; Leroy, V.; Prost-Boucle, A.; and Pétrot, F. 2017. Ternary neural networks for resource-efficient AI applications. In *2017 international joint conference on neural networks (IJCNN)*, 2547–2554. IEEE.
- Asadi, A. R.; Appiah, J.; Muntaka, S. A.; and Kropczynski, J. 2024. Actions, Not Apps: Toward Using LLMs to Reshape Context Aware Interactions in Mixed Reality Systems. In *International Conference on Human-Computer Interaction*, 166–176. Springer.
- Dettmers, T.; Lewis, M.; Belkada, Y.; and Zettlemoyer, L. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35: 30318–30332.
- Esteban-Lozano, I.; Castro-González, Á.; and Martínez, P. 2024. Using a LLM-Based Conversational Agent in the Social Robot Mini. In *International Conference on Human-Computer Interaction*, 15–26. Springer.
- Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. 2023. OPTQ: Accurate Quantization for Generative Pre-trained Transformers. In *The Eleventh International Conference on Learning Representations*.
- Hendrycks, D.; and Gimpel, K. 2016. Gaussian Error Linear Units (GELUs).
- Hubara, I.; Courbariaux, M.; Soudry, D.; El-Yaniv, R.; and Bengio, Y. 2016. Binarized neural networks. *Advances in neural information processing systems*, 29.
- Jouppi, N. 2017. Quantifying the performance of the TPU, our first machine learning chip. *Google Cloud Platform Blog, Google*, 5.
- Jouppi, N. P.; Young, C.; Patil, N.; Patterson, D.; Agrawal, G.; Bajwa, R.; Bates, S.; Bhatia, S.; Boden, N.; Borchers, A.; et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proc. of the 44th Annual Int. Symp. on Comput. Architecture*, 1–12.
- Kim, S.; Hooper, C.; Wattanawong, T.; Kang, M.; Yan, R.; Genc, H.; Dinh, G.; Huang, Q.; Keutzer, K.; Mahoney, M. W.; et al. 2023. Full stack optimization of transformer inference: a survey. *arXiv preprint arXiv:2302.14017*.
- Ma, S.; Wang, H.; Ma, L.; Wang, L.; Wang, W.; Huang, S.; Dong, L.; Wang, R.; Xue, J.; and Wei, F. 2024. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*.
- Ma, X.; Fang, G.; and Wang, X. 2023. LLM-Pruner: On the Structural Pruning of Large Language Models. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 21702–21720. Curran Associates, Inc.
- Morales, P.; and Showalter-Bucher, R. A. 2023. Towards Large Language Models at the Edge on Mobile, Augmented Reality, and Virtual Reality Devices with Unity. In *System and Architecture for Generative AI on the Edge/Mobile Platforms (SAGE)*.
- Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I.; et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8): 9.
- Reidy, B. C.; Mohammadi, M.; Elbittity, M. E.; and Zand, R. 2023. Efficient deployment of transformer models on edge tpu accelerators: A real system evaluation. In *Architecture and System Support for Transformer Models (ASSYST@ISCA 2023)*.
- Samajdar, A.; Joseph, J. M.; Zhu, Y.; Whatmough, P.; Mattina, M.; and Krishna, T. 2020. A systematic methodology for characterizing scalability of DNN accelerators using SCALE-sim. In *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 58–68. IEEE.
- Samajdar, A.; Zhu, Y.; Whatmough, P.; Mattina, M.; and Krishna, T. 2018. SCALE-Sim: Systolic CNN Accelerator Simulator. *arXiv preprint arXiv:1811.02883*.
- Sánchez Cuadrado, J.; Pérez-Soler, S.; Guerra, E.; and De Lara, J. 2024. Automating the Development of Task-oriented LLM-based Chatbots. In *Proceedings of the 6th ACM Conference on Conversational User Interfaces*, 1–10.
- Shao, W.; Chen, M.; Zhang, Z.; Xu, P.; Zhao, L.; Li, Z.; Zhang, K.; Gao, P.; Qiao, Y.; and Luo, P. 2023. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*.
- Stojkovic, J.; Choukse, E.; Zhang, C.; Goiri, I.; and Torrellas, J. 2024. Towards Greener LLMs: Bringing Energy-Efficiency to the Forefront of LLM Inference. *arXiv preprint arXiv:2403.20306*.
- Tan, Y.; Min, D.; Li, Y.; Li, W.; Hu, N.; Chen, Y.; and Qi, G. 2023. Can ChatGPT replace traditional KBQA models? An in-depth analysis of the question answering performance of the GPT LLM family. In *International Semantic Web Conference*, 348–367. Springer.
- Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Tseng, A.; Chee, J.; Sun, Q.; Kuleshov, V.; and Sa, C. D. 2024. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. *arXiv:2402.04396*.
- Ugare, S.; Suresh, T.; Kang, H.; Misailovic, S.; and Singh, G. 2024. Improving llm code generation with grammar augmentation. *arXiv preprint arXiv:2403.01632*.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Wang, H.; Ma, S.; Dong, L.; Huang, S.; Wang, H.; Ma, L.; Yang, F.; Wang, R.; Wu, Y.; and Wei, F. 2023. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*.

Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, 38087–38099. PMLR.

Xu, H.; Sharaf, A.; Chen, Y.; Tan, W.; Shen, L.; Van Durme, B.; Murray, K.; and Kim, Y. J. 2024a. Contrastive preference optimization: Pushing the boundaries of llm performance in machine translation. *arXiv preprint arXiv:2401.08417*.

Xu, Y.; Han, X.; Yang, Z.; Wang, S.; Zhu, Q.; Liu, Z.; Liu, W.; and Che, W. 2024b. OneBit: Towards Extremely Low-bit Large Language Models. *arXiv preprint arXiv:2402.11295*.

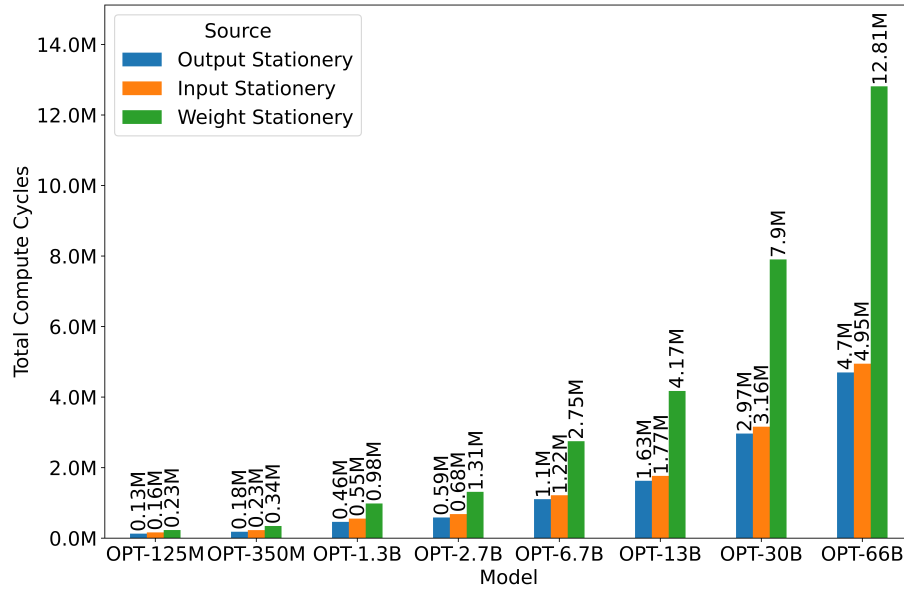
You, H.; Guo, Y.; Fu, Y.; Zhou, W.; Shi, H.; Zhang, X.; Kundu, S.; Yazdanbakhsh, A.; and Lin, Y. 2024. ShiftAddLLM: Accelerating Pretrained LLMs via Post-Training Multiplication-Less Reparameterization. *arXiv preprint arXiv:2406.05981*.

Zhang, S.; Roller, S.; Goyal, N.; Artetxe, M.; Chen, M.; Chen, S.; Dewan, C.; Diab, M.; Li, X.; Lin, X. V.; et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.

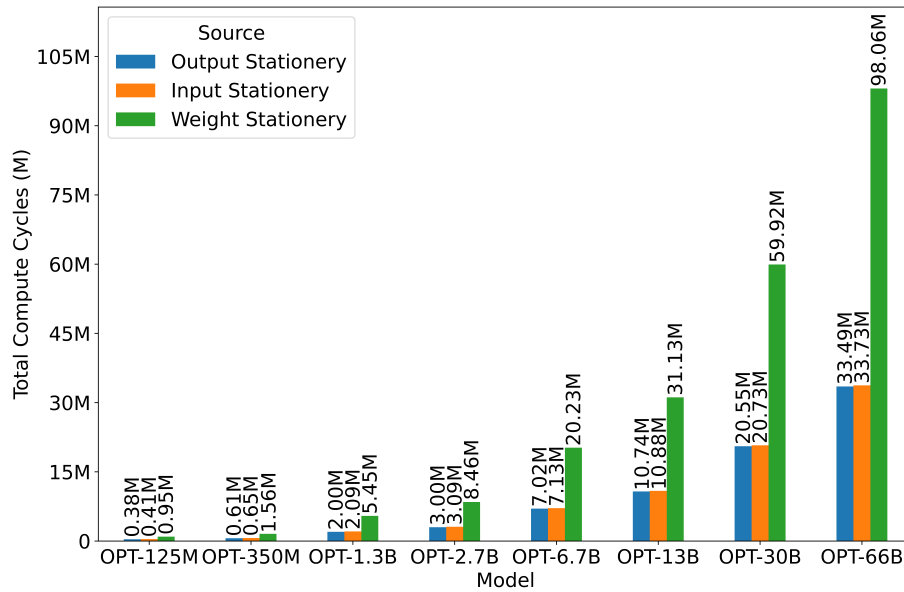
Zhu, R.-J.; Zhang, Y.; Siffrman, E.; Sheaves, T.; Wang, Y.; Richmond, D.; Zhou, P.; and Eshraghian, J. K. 2024a. Scalable MatMul-free Language Modeling. *arXiv preprint arXiv:2406.02528*.

Zhu, R.-J.; Zhang, Y.; Siffrman, E.; Sheaves, T.; Wang, Y.; Richmond, D.; Zhou, P.; and Eshraghian, J. K. 2024b. Scalable MatMul-free Language Modeling. *arXiv preprint arXiv:2406.02528*.

## Appendix A. TPU Dataflow Analysis



(a) Cloud Setup



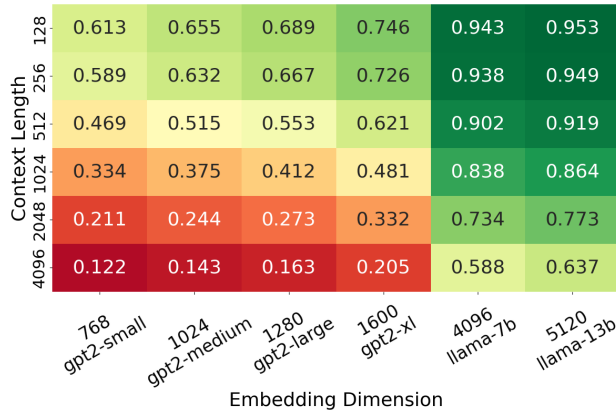
(b) Edge Setup

Figure 8: Total cycles for different dataflow of OPT models.

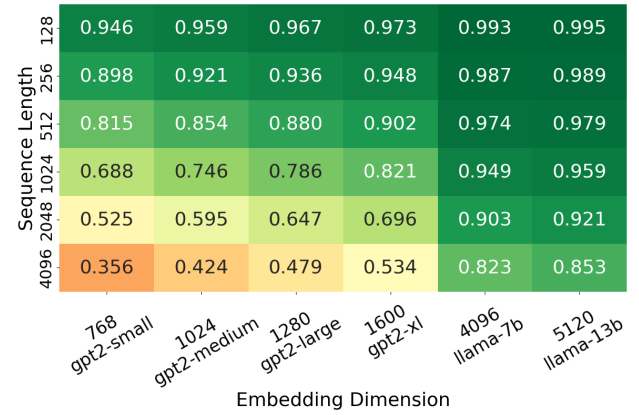
All experiments were conducted on a computing setup with a single node, 48 cores, and 200 GB of memory. The Scale-Sim V2 tool was installed, and experiments were executed using specific configurations for both cloud and edge setups.



## Appendix B. GPT and LLaMA Results

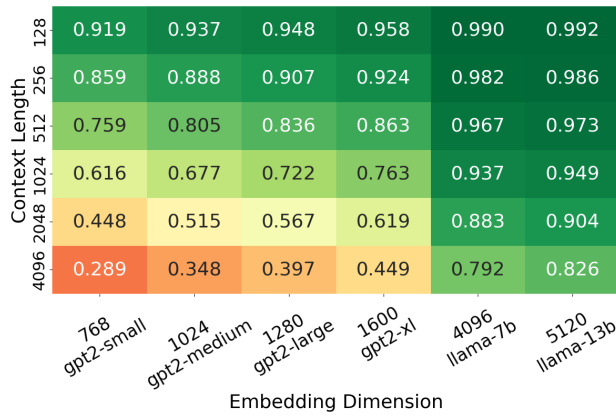


(a) Compute Cycles

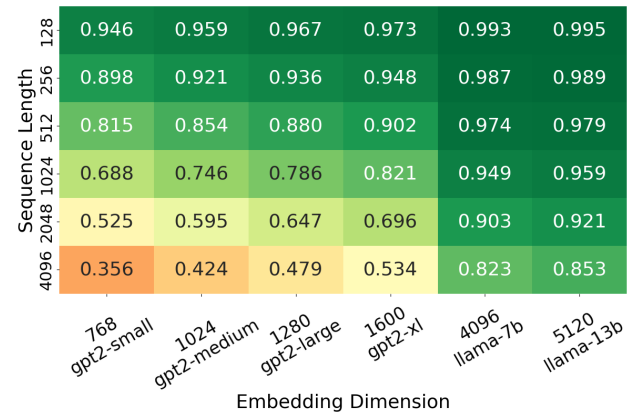


(b) Memory Access

Figure 9: Fraction of Matmul-free operations in the GPT and LLaMA models for the cloud setup.



(a) Compute Cycles

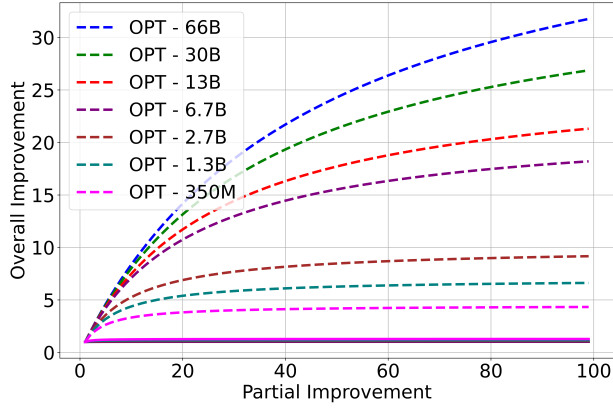


(b) Memory Access

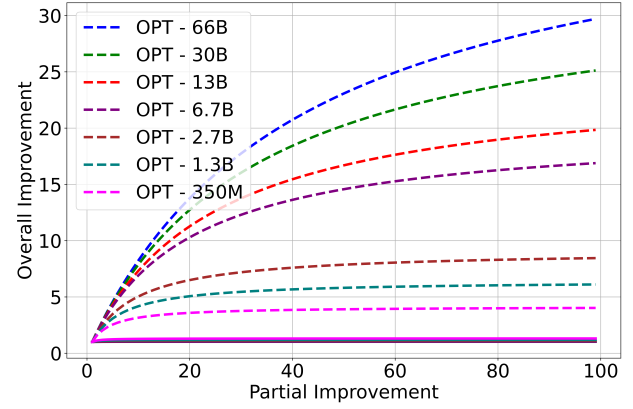
Figure 10: Fraction of Matmul-free operations in the GPT and LLaMA models for the edge setup.

All experiments were conducted on a computing setup with a single node, 48 cores, and 200 GB of memory. The Scale-Sim V2 tool was installed, and experiments were executed using specific configurations for both cloud and edge setups.

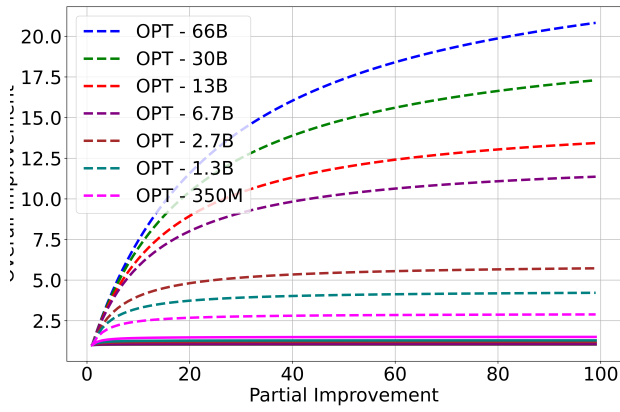
## Appendix C. Amdahl's Law Analysis



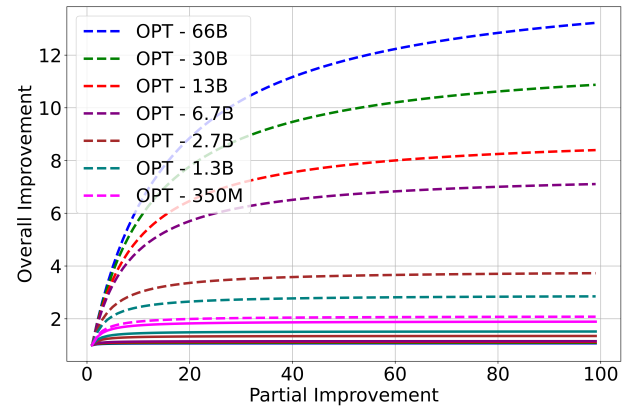
(a) Sequence Length 128



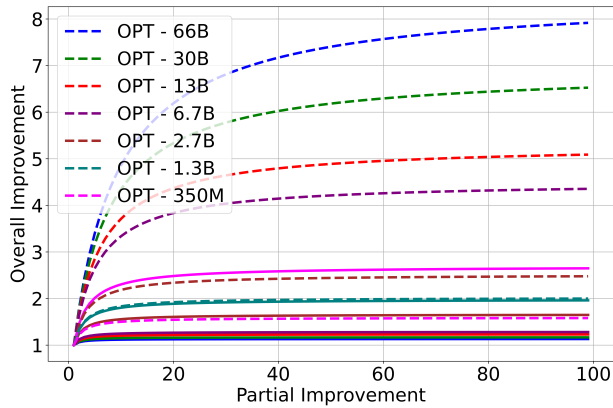
(b) Sequence Length 256



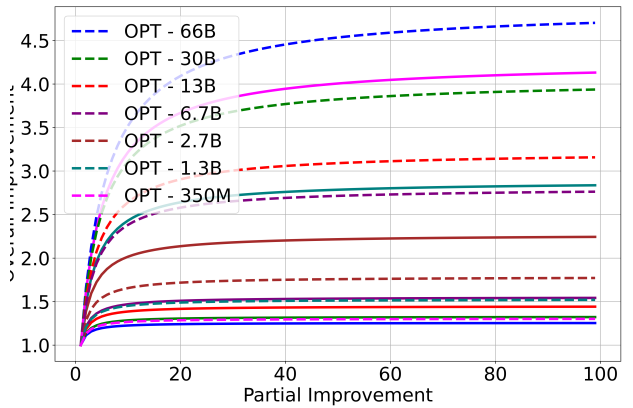
(c) Sequence Length 512



(d) Sequence Length 1024

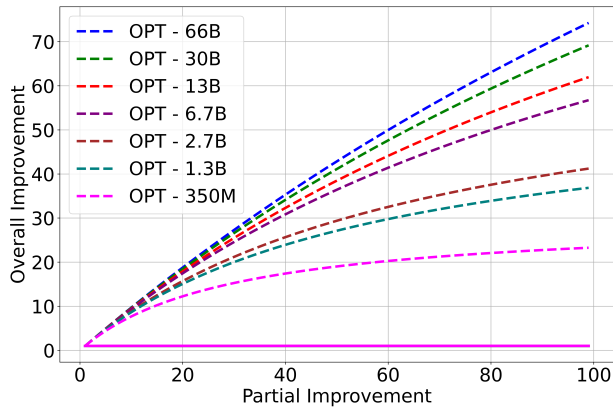


(e) Sequence Length 2048

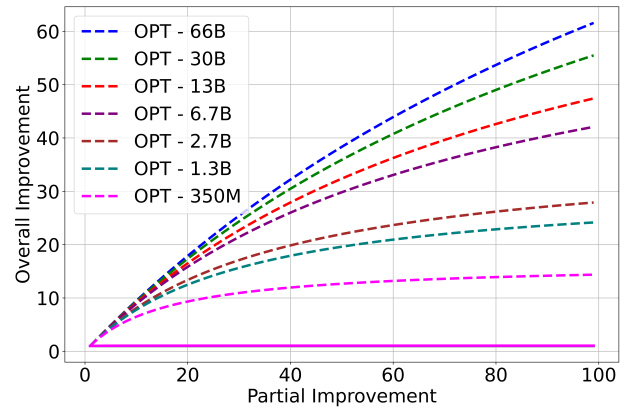


(f) Sequence Length 4096

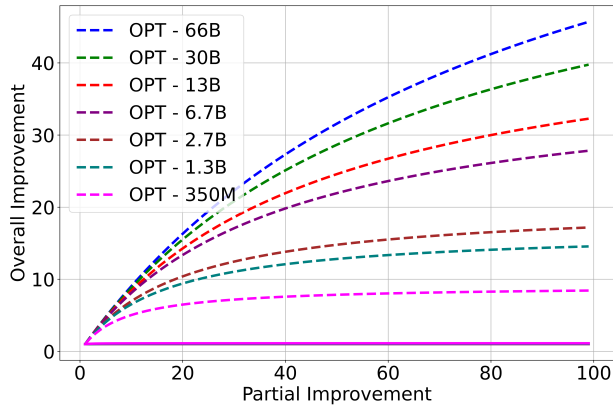
Figure 11: Amdahl's law for different sequence lengths in the OPT models for the cloud setup.



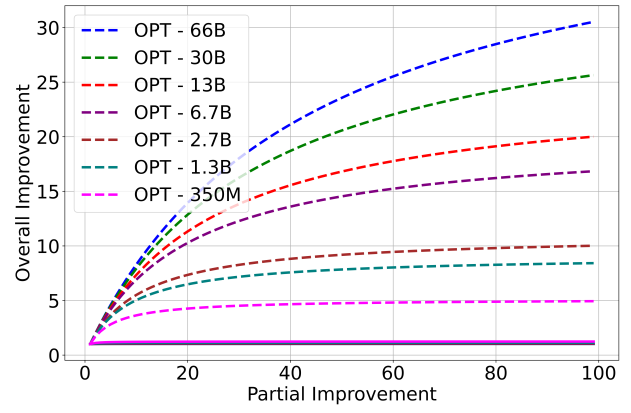
(a) Sequence Length 128



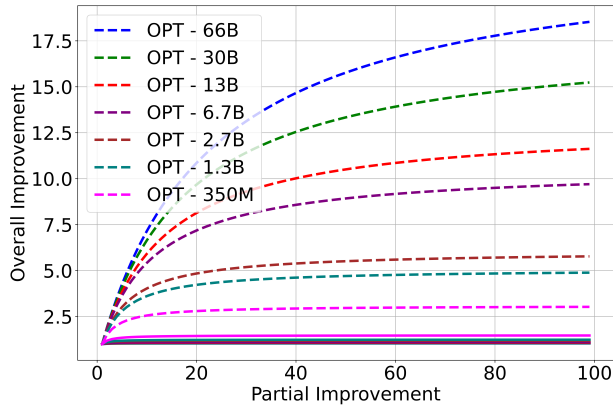
(b) Sequence Length 256



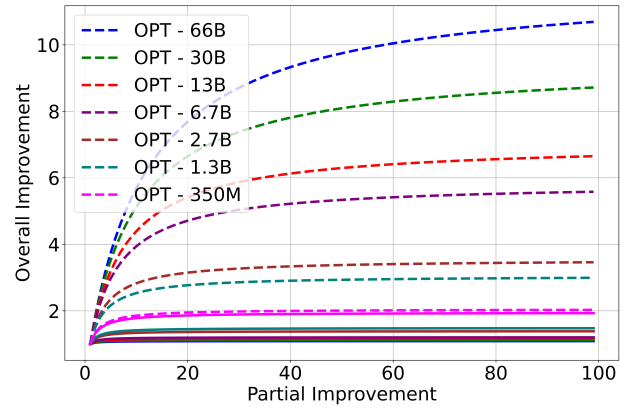
(c) Sequence Length 512



(d) Sequence Length 1024



(e) Sequence Length 2048



(f) Sequence Length 4096

Figure 12: Amdahl's law for different sequence lengths in the OPT models for the edge setup.

All experiments were conducted on a computing setup with a single node, 48 cores, and 200 GB of memory. The Scale-Sim V2 tool was installed, and experiments were executed using specific configurations for both cloud and edge setups.